

CE4052/CZ4052 Cloud Computing

Semester 2 Examination 2022-2023 — Model Answers

Question 2 uses the corrected graph provided by the user:

- $1 \rightarrow \{2, 4\}$
 - $2 \rightarrow \{4\}$
 - $3 \rightarrow \{1, 2\}$
 - $4 \rightarrow \{3\}$
-

Question 1

1(a) Two advantages of Hadoop / MapReduce for distributed systems (10 marks)

1. Scalability on commodity clusters MapReduce splits a job into many independent map tasks and reduce tasks. This allows a very large data set to be processed across many low-cost machines.

- In **word count**, each mapper processes a different input shard and emits $(w, 1)$ pairs.
- The reducers aggregate all occurrences of the same word.
- If the input doubles in size, we can usually scale out by adding more worker nodes instead of replacing one machine with a much larger one.

This is a major advantage in distributed systems because web-scale workloads often grow beyond the limits of a single server.

2. Built-in fault tolerance and simplified programming model The programmer only writes:

- $\text{map}(k, v) \rightarrow \text{list}(k_2, v_2)$
- $\text{reduce}(k_2, \text{list}(v_2)) \rightarrow \text{list}(k_3, v_3)$

The framework handles:

- task scheduling,
- data partitioning,
- shuffle/sort,
- failure recovery,
- re-execution of failed tasks,
- moving computation close to data.

Example: in **PageRank**, each iteration can be written as a MapReduce job:

- Map: page j emits contribution r_j/d_j to each outgoing neighbor.
- Reduce: sum incoming contributions to produce the next PageRank value.

If one worker crashes during a large iteration, Hadoop simply re-runs that task on another node. The user does not need to implement distributed coordination manually.

Summary Two strong advantages are:

1. **horizontal scalability** using many commodity machines,
2. **automatic fault tolerance plus a simple programming abstraction.**

These make Hadoop/MapReduce effective for applications such as word count, inverted indexing, log analysis, and PageRank.

1(b) 3-stage Close network for $N = 16$, $n = 4$, $k = 6$, $m = 4$ (15 marks)

For a 3-stage Close network $C(n, m, r)$ with total inputs $N = rn$:

$$r = N/n = 16/4 = 4$$

So the topology is:

- **Stage 1:** $r = 4$ switches, each of size $n \times k = 4 \times 6$
- **Stage 2:** $k = 6$ switches, each of size $m \times m = 4 \times 4$
- **Stage 3:** $r = 4$ switches, each of size $k \times n = 6 \times 4$

3-stage Clos Network (2023 Q1b): $N=16$, $n=4$, $k=6$, $m=4$

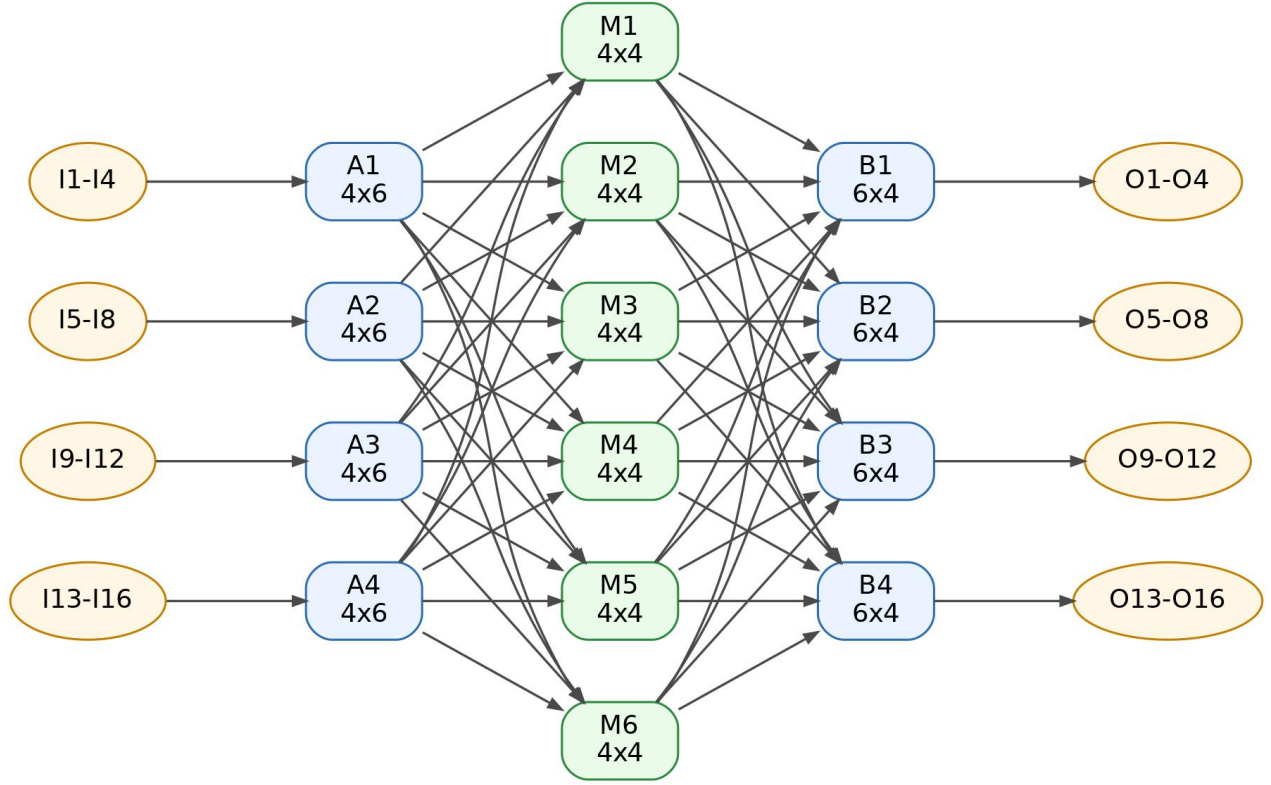


Figure 1: 3-stage Close network diagram ($N=16$, $n=4$, $k=6$, $m=4$)

One valid drawing Figure: 3-stage Close topology for the parameters in Question 1(b).

Each first-stage switch has 4 inputs and 6 outputs, one to each middle switch. Each middle switch has 4 inputs from the 4 first-stage switches and 4 outputs to the 4 third-stage switches.

Can we always add a new connection without rearrangement? No.

A 3-stage Close network is **strict-sense nonblocking** only if:

$$m \geq 2n - 1$$

Here:

$$2n - 1 = 2(4) - 1 = 7$$

but the question gives:

$$m = 4 < 7$$

So this network is **not strict-sense nonblocking**. Therefore, we **cannot** always add a new connection without **rearrangement**.

What property does it have? A Close network is **rearrangeably nonblocking** if:

$$m \geq n$$

Here $m = 4$ and $n = 4$, so this condition holds. Hence the network **can realize any valid permutation, but may require rearranging some existing connections first.**

Conclusion

- **Without rearrangement:** not always possible.
 - **With rearrangement:** yes, because $m = n$ makes it rearrangeably nonblocking.
-

Question 2

Given graph

The corrected graph is:

- $1 \rightarrow \{2, 4\}$
- $2 \rightarrow \{4\}$
- $3 \rightarrow \{1, 2\}$
- $4 \rightarrow \{3\}$

Using the standard PageRank balance equations with teleportation probability 0:

$$\begin{aligned}r_1 &= \frac{r_3}{2} \\r_2 &= \frac{r_1}{2} + \frac{r_3}{2} \\r_3 &= r_4 \\r_4 &= \frac{r_1}{2} + r_2\end{aligned}$$

with normalization:

$$r_1 + r_2 + r_3 + r_4 = 1$$

2(a) Compute the PageRank and rank the webpages (10 marks)

From the graph,

$$\begin{aligned}r_1 &= \frac{r_3}{2} \\r_2 &= \frac{r_1}{2} + \frac{r_3}{2} \\r_3 &= r_4\end{aligned}$$

Substitute $r_1 = r_3/2$ into the equation for r_2 :

$$r_2 = \frac{1}{2} \left(\frac{r_3}{2} \right) + \frac{r_3}{2} = \frac{r_3}{4} + \frac{r_3}{2} = \frac{3r_3}{4}$$

Using normalization:

$$\begin{aligned}r_1 + r_2 + r_3 + r_4 &= 1 \\ \frac{r_3}{2} + \frac{3r_3}{4} + r_3 + r_3 &= 1\end{aligned}$$

$$\left(\frac{2}{4} + \frac{3}{4} + \frac{4}{4} + \frac{4}{4}\right) r_3 = 1$$

$$\frac{13}{4} r_3 = 1$$

$$r_3 = \frac{4}{13}$$

Hence:

$$r_4 = \frac{4}{13}$$

$$r_1 = \frac{r_3}{2} = \frac{2}{13}$$

$$r_2 = \frac{3r_3}{4} = \frac{3}{13}$$

Therefore the PageRank vector is:

$$(r_1, r_2, r_3, r_4) = \left(\frac{2}{13}, \frac{3}{13}, \frac{4}{13}, \frac{4}{13}\right)$$

Ranking

$$3 = 4 > 2 > 1$$

So webpages 3 and 4 tie for the highest PageRank, followed by 2, then 1.

2(b) Teleportation probability changes to 0.1 (10 marks)

With teleportation probability 0.1, the random surfer:

- follows a hyperlink with probability 0.9,
- teleports uniformly to one of the 4 pages with probability 0.1.

A quick approximation using the result in 2(a) is:

$$r' \approx 0.9r + 0.1 \begin{bmatrix} 1/4 \\ 1/4 \\ 1/4 \\ 1/4 \end{bmatrix}$$

Using

$$r = \begin{bmatrix} 2/13 \\ 3/13 \\ 4/13 \\ 4/13 \end{bmatrix}$$

we get:

$$r'_1 \approx 0.9 \left(\frac{2}{13}\right) + 0.025 = 0.1385 + 0.025 = 0.1635$$

$$r'_2 \approx 0.9 \left(\frac{3}{13}\right) + 0.025 = 0.2077 + 0.025 = 0.2327$$

$$r'_3 = r'_4 \approx 0.9 \left(\frac{4}{13}\right) + 0.025 = 0.2769 + 0.025 = 0.3019$$

So the approximation is:

$$r' \approx (0.1635, 0.2327, 0.3019, 0.3019)$$

Comment on quality This approximation is good because the teleportation probability is still small. The exact solution is slightly different:

$$(r_1, r_2, r_3, r_4) = (0.1604, 0.2325, 0.3008, 0.3064)$$

So the approximation preserves the correct broad ordering, with pages 3 and 4 remaining the most important and page 1 remaining the least important.

Question 3

3(a) EDF with non-work-conserving VMs (10 marks)

Given five VMs:

- VM1: (1, 8, 0)
- VM2: (1, 5, 0)
- VM3: (2, 6, 0)
- VM4: (2, 10, 0)
- VM5: (2, 15, 0)

Their utilizations are:

$$u_1 = \frac{1}{8}, \quad u_2 = \frac{1}{5}, \quad u_3 = \frac{2}{6} = \frac{1}{3}, \quad u_4 = \frac{2}{10} = \frac{1}{5}, \quad u_5 = \frac{2}{15}$$

Total utilization:

$$U = \frac{1}{8} + \frac{1}{5} + \frac{1}{3} + \frac{1}{5} + \frac{2}{15}$$

Using denominator 120:

$$U = \frac{15 + 24 + 40 + 24 + 16}{120} = \frac{119}{120} < 1$$

So the set is **schedulable**.

Additional VM(s) that can still be admitted Remaining spare utilization is:

$$1 - \frac{119}{120} = \frac{1}{120}$$

Therefore any additional non-work-conserving VM with specification $(x, y, 0)$ is admissible if:

$$\frac{x}{y} \leq \frac{1}{120}$$

Equivalently:

$$y \geq 120x$$

So all possible additional single-VM specifications are:

- (1, 120, 0), (1, 121, 0), (1, 122, 0), ...
- (2, 240, 0), (2, 241, 0), (2, 242, 0), ...

- $(3, 360, 0), (3, 361, 0), \dots$
- in general, **any** $(x, y, 0)$ with $y \geq 120x$.

If multiple new VMs are admitted together, their total additional utilization must satisfy:

$$\sum_i \frac{x_i}{y_i} \leq \frac{1}{120}$$

That is the complete admission condition.

3(b) All VMs become work-conserving (10 marks)

Now the specifications are:

- VM1: $(1, 8, 1)$
- VM2: $(1, 5, 1)$
- VM3: $(2, 6, 1)$
- VM4: $(2, 10, 1)$
- VM5: $(2, 15, 1)$

The guaranteed utilizations are unchanged, so over the hyperperiod the baseline reserved CPU quanta are proportional to their utilizations.

The hyperperiod is:

$$\text{lcm}(8, 5, 6, 10, 15) = 120$$

Guaranteed CPU quanta in one hyperperiod:

VM	Utilization	CPU quantum in 120 time units
VM1	1/8	15
VM2	1/5	24
VM3	1/3	40
VM4	1/5	24
VM5	2/15	16

The total guaranteed CPU is:

$$120 \cdot \frac{119}{120} = 119$$

So there is only **1 spare quantum** in each hyperperiod that can be used as extra time in work-conserving mode.

Which gets the most CPU quantum? VM3 has the largest guaranteed share:

40 quanta

So **VM3 gets the most CPU quantum**.

Which gets the least CPU quantum? VM1 has the smallest guaranteed share:

15 quanta

So **VM1 gets the least CPU quantum**.

Explanation Work-conserving mode only changes what happens to the **single leftover quantum**. Since there is only 1 spare unit per 120-unit hyperperiod, this does not change the overall ordering of the VMs by CPU share.

Hence:

Most: VM3, Least: VM1

Question 4

4(a) CAP theorem for distributed crowdsourcing systems, using ReCAPTCHA and Google Instant Search (10 marks)

The CAP theorem states that when a **network partition** occurs, a distributed system cannot simultaneously guarantee all three of:

- **Consistency (C)**: all users see the same latest state,
- **Availability (A)**: every request gets a response,
- **Partition tolerance (P)**: the system continues operating despite communication failure between nodes.

In practice, a large internet-scale system must tolerate partitions, so the real design tradeoff is usually between **consistency** and **availability**.

Applying CAP to crowdsourcing systems A distributed crowdsourcing system coordinates many geographically distributed users who submit responses, labels, clicks, or human-verification results. During a partition, the system can either:

1. **Prefer consistency (CP)**: block or delay some requests to avoid conflicting global state.
2. **Prefer availability (AP)**: accept responses everywhere and reconcile conflicts later.

ReCAPTCHA ReCAPTCHA is a good crowdsourcing example because users solve challenges, and the collected human responses help classify content or improve models.

A ReCAPTCHA service usually favors **availability + partition tolerance** for challenge delivery:

- the service should still present a challenge quickly,
- it should still accept human responses from many regions,
- temporary replica inconsistency is acceptable because answers can be aggregated or reconciled later.

So ReCAPTCHA is typically **AP-leaning** in its data collection path.

However, for security-sensitive decisions such as fraud detection or abuse scoring, some internal components may use **CP-like behavior** to avoid accepting contradictory or duplicated trusted state.

Google Instant Search Google Instant Search is also strongly **AP-leaning**:

- users expect extremely low latency,
- the service must remain responsive worldwide,
- slightly stale rankings or suggestions are acceptable for a short time.

If a partition happens, it is better to return slightly outdated suggestions than to block all search suggestions. That means the system prioritizes **availability and partition tolerance**, while allowing weaker consistency.

Conclusion For distributed crowdsourcing systems:

- If the main goal is fast response and large-scale participation, the design tends to be **AP**.
- If the main goal is strict correctness of shared global state such as payments, one-time task assignment, or exact accounting, the design tends to be more **CP**.

Using the examples:

- **ReCAPTCHA**: mostly AP for challenge serving and answer collection.
 - **Google Instant Search**: strongly AP because responsiveness is more important than perfectly synchronized state.
-

4(b) Consistent hashing event trace and load balance (15 marks)

The OCR of the question contains typographical errors, but the pattern matches the other exam papers. I use:

- $M = 11$
- server replica hashes:

$$f(ip) = ip \bmod M$$

$$g(ip) = ((ip + 51) \cdot 150) \bmod M$$

- data replica hashes:

$$f(d) = d \bmod M$$

$$g(d) = (d + 51) \bmod M$$

Step 1: Server positions on the ring Since $150 \bmod 11 = 7$:

Server s_1 with $ip = 1$

$$f(1) = 1$$

$$g(1) = ((1 + 51) \cdot 150) \bmod 11 = (52 \cdot 150) \bmod 11$$

$$52 \bmod 11 = 8, \quad 8 \cdot 7 = 56 \equiv 1 \pmod{11}$$

So s_1 is at positions **1 and 1**.

Server s_2 with $ip = 40$

$$f(40) = 40 \bmod 11 = 7$$

$$g(40) = ((40 + 51) \cdot 150) \bmod 11 = (91 \cdot 150) \bmod 11$$

$$91 \bmod 11 = 3, \quad 3 \cdot 7 = 21 \equiv 10 \pmod{11}$$

So s_2 is at positions **7 and 10**.

Server s_3 with $ip = 80$

$$f(80) = 80 \bmod 11 = 3$$

$$g(80) = ((80 + 51) \cdot 150) \bmod 11 = (131 \cdot 150) \bmod 11$$

$$131 \bmod 11 = 10, \quad 10 \cdot 7 = 70 \equiv 4 \pmod{11}$$

So s_3 is at positions **3 and 4**.

Final active servers The event sequence includes: start s_1 , start s_2 , later start s_3 , then s_3 fails.

So at the **end**, the active ring contains only:

- s_1 at position 1
- s_2 at positions 7 and 10

Step 2: Final data hash positions For each data item, compute two replicas:

Data item	$f(d) = d \bmod 11$	$g(d) = (d + 51) \bmod 11$
$d = 10$	10	6
$d = 20$	9	5
$d = 30$	8	4
$d = 40$	7	3
$d = 60$	5	1
$d = 70$	4	0
$d = 80$	3	10

Step 3: Assign each replica clockwise on the final ring Final ring positions:

- position 1: s_1
- position 7: s_2
- position 10: s_2

Therefore:

- positions 0 and 1 map to s_1
- positions 2 through 7 map to s_2
- positions 8 through 10 map to s_2

So the final responsibilities are:

Data item	Replica positions	Responsible server(s)
$d = 10$	10, 6	s_2, s_2
$d = 20$	9, 5	s_2, s_2
$d = 30$	8, 4	s_2, s_2
$d = 40$	7, 3	s_2, s_2
$d = 60$	5, 1	s_2, s_1
$d = 70$	4, 0	s_2, s_1
$d = 80$	3, 10	s_2, s_2

Is the load balanced at the end? No.

At the end:

- s_1 covers only the short arc ending at position 1,
- s_2 covers almost the entire ring using positions 7 and 10,
- s_1 also suffers because both of its virtual replicas collide at the same position 1.

Counting final replicas:

- s_1 stores only **2** replicas
- s_2 stores **12** replicas

So the load is highly skewed toward s_2 .

Conclusion The load is **not balanced** at the end because:

1. one server failed (s_3),
2. s_1 's two virtual replicas collapse to the same position,
3. the remaining active server positions are unevenly spread on the ring.

4(c) Probability that n random points lie on a semicircle, and effect on consistent hashing (10 marks)

For n points placed independently and uniformly at random on a circle, the probability that **all n points lie within some semicircle** is:

$$\Pr(\text{all } n \text{ points lie on a semicircle}) = \frac{n}{2^{n-1}}$$

Why? A standard argument is:

- Pick the leftmost point of the semicircle boundary among the n points.
- For all points to fit in a semicircle starting at that point, the remaining $n - 1$ points must all fall within the next half-circle.
- Each remaining point independently has probability $1/2$ of landing in that semicircle.
- There are n choices for which point is the boundary point.

Hence:

$$n \left(\frac{1}{2} \right)^{n-1} = \frac{n}{2^{n-1}}$$

Effect on consistent hashing load distribution In consistent hashing, server positions are random points on a ring. If many points happen to lie within a semicircle, then:

- the opposite half of the ring contains very few or no servers,
- one or a few servers will own a very large arc,
- many data items mapped into that large arc will concentrate on those servers.

So this probability measures one form of **load imbalance risk**.

As n increases,

$$\frac{n}{2^{n-1}} \rightarrow 0$$

very quickly, which means severe one-sided clustering becomes less likely. This is why using **many virtual nodes** improves balance in consistent hashing: more random points on the ring reduce the chance of large empty arcs and smooth out the load distribution.

End of 2023 Model Answers